HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I

BÁO CÁO HẰNG TUẦN MÔN
THỰC TẬP CƠ SỞ

Giảng viên hướng dẫn : TS. Kim Ngọc Bách
Sinh viên thực hiện : Mã sinh viên
Nguyễn Việt Quang : B23DCCN691

Hà Nội – 2026

I. Triển khai huấn luyện Full Finetuning trên nền tảng Cloud và hoàn thiện Pipeline đánh giá mô hình

1. Mở đầu

Tiếp nối thành công của việc chạy thử nghiệm (dry-run) pipeline huấn luyện ở tuần trước, mục tiêu tuần này là tiến hành huấn luyện chính thức (Full Finetuning) trên toàn bộ tập dữ liệu UIT-OpenViIC. Tuy nhiên, trong quá trình thực nghiệm trên thiết bị cá nhân (GPU NVIDIA GeForce GTX 1650 - 4GB VRAM), một rào cản lớn về phần cứng đã xuất hiện: thời gian để hoàn thành một epoch lên tới khoảng 10 tiếng đồng hồ. Với cấu hình dự kiến là 10 đến 20 epochs, việc huấn luyện cục bộ sẽ làm chậm trễ nghiêm trọng tiến độ của đề tài và không cho phép thử nghiệm các siêu tham số (hyperparameters) khác nhau.

Bên cạnh đó, pipeline xây dựng ở tuần trước mới chỉ dừng lại ở bước tối ưu hàm Loss trên tập Train (Training Loop) mà chưa có cơ chế kiểm định (Validation), suy luận (Test) và đo lường các thang đo học thuật (Metrics) để chứng minh chất lượng thực sự của mô hình.

Vì vậy, trong tuần này, nội dung công việc được chuyển hướng sang việc di chuyển toàn bộ hệ thống lên nền tảng đám mây (Kaggle Cloud) để tận dụng sức mạnh GPU lớn hơn. Đồng thời, em tiến hành nâng cấp mã nguồn, bổ sung hoàn thiện vòng lặp Validation, Test và tích hợp các công cụ tính toán metric chuyên dụng cho bài toán Image Captioning.

2. Mục tiêu thực hiện Mục tiêu cụ thể của tuần này bao gồm ba phần chính:

- Phần thứ nhất: Chuyển đổi môi trường huấn luyện lên nền tảng Kaggle (sử dụng GPU Tesla T4 16GB VRAM), thiết lập lại Dataloader để tối ưu hóa tốc độ nạp dữ liệu.
- Phần thứ hai: Xây dựng Validation Loop kết hợp với cơ chế Checkpointing nhằm lưu lại trọng số (weights) tốt nhất của mô hình dựa trên Validation Loss, tránh hiện tượng học vẹt (Overfitting).
- Phần thứ ba: Xây dựng Test Loop phục vụ quá trình suy luận (Inference) bằng thuật toán Beam Search, đồng thời tích hợp các thang đo chuẩn trong NLP gồm BLEU, ROUGE và METEOR để đánh giá chất lượng câu mô tả sinh ra.

3. Môi trường và thiết lập thực hiện

Do tính chất đòi hỏi tài nguyên tính toán lớn, môi trường thực hiện tuần này được chuyển sang nền tảng Kaggle Notebook.

Phần cứng: GPU NVIDIA Tesla T4 với 16GB VRAM, cho phép tăng kích thước batch size đáng kể so với máy cá nhân.

Công cụ và Thư viện: Python 3.12, PyTorch, Transformers (Hugging Face). Để tính toán các thang đo đánh giá, thư viện evaluate và rouge_score được cài đặt bổ sung.

4. Nội dung thực hiện chi tiết Quá trình hoàn thiện pipeline được chia thành các phân hệ logic như sau:

4.1. Tối ưu Dataloader và xử lý lỗi dữ liệu trên Cloud

Nhờ nâng cấp lên GPU Tesla T4 16GB, biến BATCH_SIZE đã được tăng từ 2 lên 64, giúp quá trình tính toán ma trận trên GPU hiệu quả hơn rất nhiều. Tuy nhiên, khi chạy thực tế trên toàn bộ tập dữ liệu, hệ thống đã phát sinh lỗi làm gián đoạn tiến trình do một số file ảnh bị hỏng (cảnh báo Premature end of JPEG file và libpng warning). Để giải quyết triệt để nút thắt này, em đã cập nhật lại phương thức __getitem__ trong lớp UITViICDataset bằng cách bổ sung khối lệnh try-except. Khi hệ thống gặp ảnh lỗi (hàm cv2.imread trả về None), nó sẽ tự động bỏ qua và lấy ngẫu nhiên một mẫu dữ liệu hợp lệ khác để thay thế, đảm bảo pipeline không bao giờ bị crash giữa chừng.

4.2. Bổ sung Validation Loop và Cơ chế Checkpointing

Để giám sát mô hình không bị Overfitting, em bổ sung một vòng lặp Validation diễn ra ngay sau mỗi epoch huấn luyện. Ở pha này, mô hình được chuyển sang chế độ model.eval(), các gradient không được tính toán (torch.no_grad()). Cơ chế Checkpointing cũng được thiết lập: một biến best_val_loss được khởi tạo bằng vô cực. Sau mỗi epoch, nếu Loss trung bình trên tập dev_dataloader thấp hơn kỷ lục cũ, hệ thống sẽ tự động gọi hàm model.save_pretrained() để lưu trữ trạng thái tốt nhất này vào thư mục ./saved_models.

4.3. Xây dựng Test Loop và tính toán Metrics

Khác với Train/Val tính toán Language Modeling Loss, pha Test sử dụng hàm .generate() để mô hình tự động sinh ra các chuỗi token dựa vào ảnh đầu vào. Để tăng chất lượng văn bản, thuật toán num_beams=3 (Beam Search) và

early_stopping=True được áp dụng. Đầu ra sau đó được đưa qua processor.batch_decode để dịch ngược thành văn bản tiếng Việt. Cuối cùng, các câu dự đoán (Predictions) này được đối chiếu với câu gốc (References) thông qua thư viện evaluate của Hugging Face để tính toán 3 thang đo:

BLEU: Đánh giá độ chính xác của các cụm từ (N-gram precision).

ROUGE-L: Đánh giá độ phủ dựa trên chuỗi con chung dài nhất (Longest Common Subsequence).

METEOR: Đánh giá mức độ tương đồng linh hoạt hơn.

5. Kết quả và Nhận xét

Về hiệu năng phần cứng: Việc chuyển dịch sang Kaggle với batch size 64 đã cải thiện tốc độ rõ rệt. Tuy nhiên, thời gian trung bình cho mỗi iteration rơi vào khoảng ~6.4s, đồng nghĩa một epoch vẫn mất khoảng 1 giờ để hoàn thành. Dù vậy, mức này đã khả thi hơn rất nhiều so với mốc 10 giờ trên máy cá nhân.

Về tính toàn vẹn của mã nguồn: Code mới đã tự động bỏ qua thành công các bức ảnh bị lỗi nén (JPEG/PNG corruption) mà không làm sập hệ thống. Vòng lặp Validation hoạt động trơn tru và đã có thể lưu lại các checkpoint đầu tiên.

6. Kết luận

Trong tuần này, em đã giải quyết thành công bài toán nghẽn cổ chai về phần cứng bằng cách chuyển môi trường huấn luyện lên nền tảng đám mây Kaggle. Đồng thời, cấu trúc của mã nguồn huấn luyện đã được hoàn thiện 100% cho một quy trình Deep Learning tiêu chuẩn: từ xử lý ngoại lệ dữ liệu, Training, Validation, Checkpointing cho đến Testing và đo lường Metric (BLEU, ROUGE, METEOR). Đây là bước đệm cuối cùng vững chắc. Trong những ngày tới, em sẽ tiến hành treo máy huấn luyện (Full Training) cho đến khi mô hình hội tụ hoàn toàn, sau đó trích xuất các chỉ số Metric cuối cùng để chuẩn bị cho báo cáo tổng kết đề tài.